

CASO DE ESTUDIO: GUARDIAN OF MISSING

DOCUMENTO DE INGENIERÍA DE SOFTWARE - UNIDAD I: DEFINICIÓN DEL PROCESO DE DESARROLLO WEB

Carrera: Ingeniería en Desarrollo y Gestión de Software

Asignatura: Desarrollo Web Integral (Noveno Cuatrimestre)

Proyecto: Ecosistema de Seguridad Personal "Guardian of Missing"

Equipo de Desarrollo: Equipo MR (Erick Matias Granillo Mejia ,Mauricio Rosales Gabriel , Diego Miguel Rivera Chavez, Derek Sesni Carreño, José Arturo Garcia Gonzalez)

CAPÍTULO I: PLAN DEL PROCESO DE DESARROLLO (METODOLOGÍA ÁGIL)

1.1 Selección de la Metodología Ágil: Scrum

Para el desarrollo del ecosistema integral "Guardian of Missing", se ha seleccionado la metodología ágil **Scrum**. Esta elección se justifica debido a la naturaleza crítica, adaptativa y multiplataforma del proyecto, el cual involucra una aplicación móvil, una aplicación para smartwatch y una API web centralizadora. Dado que existen puntos pendientes de definición técnica complejos —como la frecuencia del rastreo GPS (polling) y la viabilidad de alarmas en altavoces de los wearables—, Scrum permite realizar entregas parciales y funcionales (Incrementos) al finalizar cada ciclo de trabajo (Sprint), facilitando la reevaluación constante de requerimientos y la mitigación de riesgos técnicos tempranos sin detener el avance general del proyecto.

1.2 Planificación de Sprints y Cronograma Inicial

El desarrollo se estructurará en Sprints que su duración dependerá de la dificultad y complejidad de cada uno, acumulando el total de horas prácticas y teóricas estipuladas para el análisis y cimentación del sistema.

Sprints	Objetivo Principal	Entregables y Componentes Incluidos
Sprint 1 (04 May – 17 May, 2026)	Análisis de Requisitos y Arquitectura Inicial.	* Documentación: Levantamiento de la visión general del proyecto, especificación de módulos core (Módulo 1 y Módulo 2) y diagramación del ecosistema (Smartwatch + Móvil + Web).
Sprint 2 (18 May – 31 May, 2026) <i>Nota: Documentación cierra el 24 de Mayo.</i>	Cierre de Documentación, Contratos de API y Despliegue de Base de Datos.	* Documentación (Cierre al 24 de mayo): DOC-01 a DOC-03. Firma de contratos API (Swagger) y resolución de <i>Action Items</i> (polling de GPS, algoritmos de moderación y potencia de altavoces). * Bases de Datos: DB-01 a DB-04. Scripts de bases de datos relacionales (usuarios/roles), configuración geoespacial (PostGIS/MongoDB), clúster de caché en memoria (Redis) e infraestructura en la nube (AWS S3) para evidencia.
Sprint 3 (01 Jun – 14 Jun, 2026)	Motor de Geolocalización Core y Renderizado de Mapas.	* Backend: BE-01. Desarrollo del motor de procesamiento de coordenadas (GPS Engine), lógica de las 3 reglas de la "Última Ubicación Conocida" y control condicional de <i>polling</i> (24/7 vs On-Demand). * Frontend: FE-01. Interfaz adaptativa del mapa con

capas de mapas de calor (Móvil/Web) y sistema de alertas optimizado para la pantalla del Smartwatch.

Sprint 4

(15 Jun – 28 Jun, 2026)

Núcleo de Emergencia (Botón de Pánico) y Comunicación en Tiempo Real.

* **Backend:** BE-02. Orquestador en tiempo real mediante WebSockets, lógica de cálculo de "Redes Cercanas" en milisegundos e integración con pasarelas de Notificaciones Push masivas.

* **Frontend:** FE-03. Desarrollo del panel interactivo del Botón de Pánico en celular y reloj, control remoto de cámara y switch para la "Alarma Sonora Opcional".

Sprint 5

(29 Jun – 12 Jul, 2026)

Funciones Avanzadas de Seguridad (Geocercas, Evidencia Silenciosa y Moderación).

* **Frontend:** FE-02 y FE-04. Panel de control web/móvil del cuidador para delimitación geométrica de Geocercas (Módulo 1) y formulario de Reportes Ciudadanos Anónimos (Módulo 2).

* **Backend:** BE-03 y BE-04. API de streaming en segundo plano para "Grabación Silenciosa" (audio/video) y microservicio con filtros anti-spam/difamación para reportes anónimos.

Sprint 6

Pruebas de Integración Cruzada (E2E) y Estabilización del Sistema.

* **Test y Control de Calidad:** TEST-01 y TEST-02. Pruebas exhaustivas de comunicación cruzada (Reloj ↔ Celular ↔

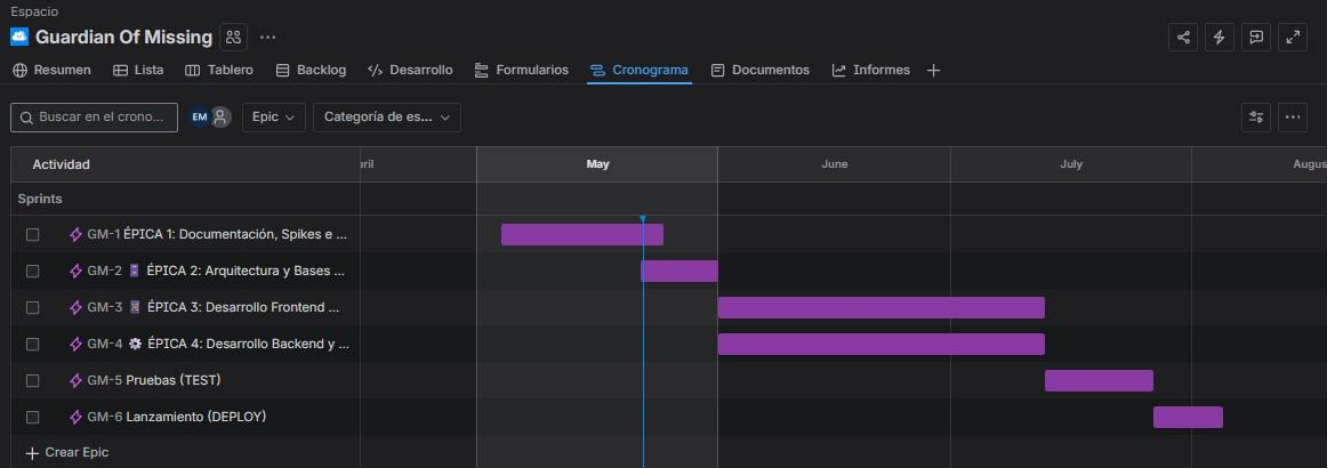
(13 Jul – 26 Jul, 2026)		Web). Pruebas de estrés y optimización de consumo de batería/RAM en segundo plano para iOS y Android. Ajustes de conectividad Bluetooth y corrección de bugs críticos.
Fase de Cierre	Despliegue Final en Producción y Aprobación en Tiendas.	* Lanzamiento (Deploy): DEPLOY-01. Preparación y subida de paquetes a Google Play Console y Apple App Store Connect. Gestión del periodo de revisión de las tiendas (validación de permisos de rastreo 24/7). Paso del código backend y bases de datos a servidores de producción estables. Cierre oficial del proyecto.
(27 Jul – 04 Ago, 2026)		

1.3 Roles y Responsabilidades (Equipo MR)

Integrante	Matrícula	Rol en el Equipo	Contacto	Descripción de Responsabilidades (Foco Técnico)
Derek Sesni Carreño	230892	Scrum Master / Lead Backend Developer	@DevFntxy	* Gestión Ágil: Remoción de impedimentos, control de tiempos en el roadmap y priorización del Backlog.* Core Backend: Arquitectura de servidores, orquestación del Botón de Pánico mediante WebSockets y control de concurrencia para alertas masivas en tiempo real.
Diego Miguel Rivera Chavez	230260	Lead Frontend Developer & DB Co-Designer	@DiegoMiguel04	* Desarrollo Multiplataforma: UI/UX adaptativa de la App Móvil, Smartwatch y Web. Implementación de los mapas de calor y la lógica del botón en el reloj.* Sincronización: Enlace de la base de datos con el consumo de estados en el Frontend.

José Arturo García González	230629	Database Administrator (DBA) & QA	@ppyo1234	* Diseño e Infraestructura de BD: Modelado relacional de usuarios y migración geoespacial (PostGIS/MongoDB) para el manejo de Geocercas y ubicaciones.* Seguridad de Evidencia: Configuración y encriptación del almacenamiento en la nube para audio/video silencioso.
Mauricio Rosales Gabriel	220859	Backend Developer (Geolocalización & API)	@elmau0834x	* Lógica de Ubicación: Programación del motor de Polling GPS de alto rendimiento, optimización de consumo de batería y backend para las 3 reglas estrictas de la "Última Ubicación Conocida" (Módulo 1).
Erick Matias Granillo Mejia	230045	Backend Developer (Seguridad & Algoritmos)	@EMATIAS	* Filtros e Ingesta: Desarrollo del algoritmo y lógica de moderación anti-spam para los reportes ciudadanos anónimos (Módulo 2).* Captura Silenciosa: Implementación del endpoint receptor de streaming discreto de audio y video en segundo plano.

Diagrama de Gant:



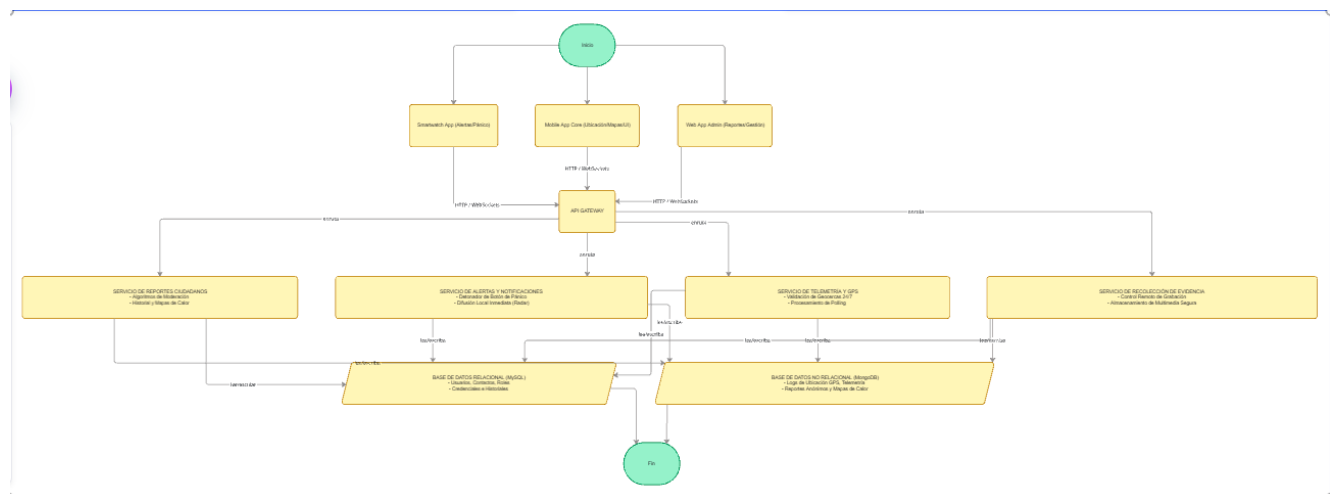
CAPÍTULO II: ESPECIFICACIÓN Y ARQUITECTURA DEL SOFTWARE

2.1 Justificación de la Arquitectura Seleccionada

Para "Guardian of Missing" se ha adoptado un modelo arquitectónico de **Microservicios e interfaces basadas en API RESTful asíncronas**, complementado internamente con una **Arquitectura Limpia (Clean Architecture)** en el núcleo del backend web. Esta decisión responde directamente a los atributos de calidad requeridos:

- **Disponibilidad y Concurrencia de Alta Densidad:** El sistema recibirá constantes ráfagas de datos de ubicación GPS procedentes de los dispositivos del Módulo 1. Separar la ingesta de telemetría de ubicación de la capa de visualización web de reportes garantiza que un colapso en las consultas web no impida la recepción de un Botón de Pánico crítico.
- **Escalabilidad Multiplataforma:** Al centralizar las reglas de negocio en servicios desacoplados, la app móvil, el reloj y la plataforma web consumen los mismos endpoints, agilizando la integración y el mantenimiento.

2.2 Esquema Conceptual de la Arquitectura del Sistema



CAPÍTULO III: PROPUESTA DE PATRONES DE DISEÑO

Para resolver los problemas específicos planteados en las reglas de negocio de la aplicación, se incorporan tres patrones de diseño de software fundamentales, cumpliendo rigurosamente la directiva de emplear un patrón creacional, uno estructural y uno de comportamiento.

3.1 Patrón Creacional: Factory Method

- **Problema a resolver:** El sistema maneja diferentes variantes de eventos de seguridad y reportes que poseen comportamientos, flujos de validación y ciclos de vida divergentes (Reporte de Persona Desaparecida del Módulo 1 frente a Reporte Ciudadano Anónimo del Módulo 2).

- **Solución en el proyecto:** Se implementará una clase abstracta `CreadorDeReportes` con un método de fabricación. Dependiendo del contexto o la carga útil recibida en el endpoint de la API, las subclases concretas instanciarán un `ReporteModulo1` (que requiere vincular metadatos de permisos fijos de GPS y validación de identidad) o un `ReporteModulo2` (que activa automáticamente los Algoritmos de Moderación y filtros contra spam/difamación antes de guardarse en MongoDB).

3.2 Patrón Estructural: Facade (Fachada)

- **Problema a resolver:** La activación del Botón de Pánico desde un Smartwatch o teléfono celular requiere la orquestación coordinada de múltiples subsistemas complejos del Backend de manera atómica: persistir la alerta en MySQL, recuperar los tokens de notificaciones push de los contactos de confianza, consultar las coordenadas de los usuarios locales en un rango cercano e inicializar la transmisión del buffer para la Grabación Silenciosa.

- **Solución en el proyecto:** Se diseña la clase fachada `FachadaEmergenciaWeb`. El controlador de la API solo realiza una llamada simple a `FachadaEmergenciaWeb.dispararAlertaCompleta(userId, coordenada)`. Esta fachada se encarga internamente de comunicarse de manera transparente con el Servicio de Notificaciones, el Servicio de Grabación y el Buscador Espacial de Coordenadas, aislando la complejidad técnica de los clientes.

3.3 Patrón de Comportamiento: Observer (Observador)

- **Problema a resolver:** Cuando se vulnera una Geocerca del Módulo 1 (un adulto mayor o paciente con Alzheimer abandona el rango seguro), múltiples entidades interesadas (la aplicación del cuidador principal, el centro de monitoreo web y la base de datos distribuida) deben reaccionar inmediatamente en tiempo real sin que el módulo de rastreo GPS esté acoplado directamente a ellas.

- **Solución en el proyecto:** El motor de Geocercas actuará como el **Sujeto (Subject)**. Diversos componentes se registrarán como **Observadores (Observers)**, tales como el ServicioNotificacionesPush y el ServicioRegistroAuditoria. En el momento exacto en que la telemetría rompe el polígono configurado, el Sujeto ejecuta un bucle de notificación automática, actualizando en milisegundos las interfaces de los cuidadores y activando los protocolos correspondientes.

3.4 Patrón Web / Emergente: BFF (Backend For Frontend)

- **Problema a resolver:** El ecosistema de software está compuesto por tres clientes con capacidades de visualización, hardware, batería y ancho de banda radicalmente divergentes (Smartwatch, Aplicación Móvil y Plataforma Web). Si todos consumieran la misma API monolítica o los mismos endpoints crudos de la base de datos, se generaría un consumo ineficiente de batería en los wearables (*over-fetching* de datos) y las interfaces adaptativas tendrían problemas para procesar estructuras de datos JSON pesadas y unificadas.
- **Solución en el proyecto:** Se implementará el patrón **BFF (Backend For Frontend)**. En lugar de tener una sola API general, se diseñarán capas de orquestación intermedias específicas para cada tipo de dispositivo:
 - **BFF-Wearable:** Filtrará las respuestas del backend para enviar estructuras JSON ultra compactas que solo contengan texto plano y alertas de color, reduciendo drásticamente el procesamiento y el consumo de datos/batería en el Smartwatch.
 - **BFF-Mobile:** Gestionará el flujo pesado de telemetría, el streaming en segundo plano de la "Grabación Silenciosa" y las coordenadas del mapa adaptativo.
 - **BFF-Web:** Diseñado específicamente para el panel del cuidador, devolviendo datos consolidados de auditoría, historiales geoespaciales y herramientas de renderizado masivo para los mapas de calor en pantallas grandes.

CAPÍTULO IV: SELECCIÓN Y JUSTIFICACIÓN DE FRAMEWORKS

4.1 Framework de Backend: FastAPI (Python)

Para el núcleo de servicios web integrales se ha seleccionado **FastAPI** sobre otras alternativas basadas en lenguajes tradicionales. Los fundamentos técnicos de esta selección son:

- **Soporte Asíncrono Nativo (async/await):** Fundamental para gestionar eficientemente las peticiones concurrentes de ubicación en tiempo real (polling continuo) sin bloquear los hilos del servidor de aplicaciones.
- **Validación de Datos con Pydantic:** Permite estructurar de forma estricta los esquemas JSON de intercambio de datos (como las reglas de la última ubicación conocida), rechazando peticiones malformadas de inmediato y garantizando seguridad en la capa de transporte.
- **Rendimiento Elevado:** Al estar construido sobre Starlette y Uvicorn, ofrece velocidades equiparables a NodeJS y Go, ideales para un ecosistema de seguridad de respuesta inmediata.

4.2 Enfoque Híbrido de Almacenamiento: MySQL y MongoDB

- **MySQL:** Se utilizará para gestionar los datos de alta integridad referencial del ecosistema, como tablas de usuarios, roles de seguridad, credenciales hash, emparejamiento de dispositivos y relaciones estrictas de contactos de confianza y asignación de cuidadores.
- **MongoDB:** Su naturaleza no relacional orientada a documentos es perfecta para almacenar el volumen masivo de logs e históricos de ubicación GPS generados 24/7 por el Módulo 1. Asimismo, permite estructurar de forma flexible los reportes ciudadanos anónimos del Módulo 2 y procesar de manera optimizada los índices geoespaciales para renderizar los mapas de calor de zonas de riesgo.

4.3 Framework/Librería de Frontend: React y React Native (JavaScript/TypeScript)

Para el desarrollo de las interfaces de usuario del ecosistema (Plataforma Web, Aplicación Móvil y Aplicación para Smartwatch), se ha seleccionado el entorno de **React**, complementado con **React Native** para el despliegue nativo en dispositivos móviles y wearables. Los fundamentos técnicos de esta selección son:

- **Arquitectura Basada en Componentes Reutilizables:** Permite diseñar componentes modulares e independientes (como el Botón de Pánico, los visores de mapas o los formularios de reportes). Estos componentes se pueden compartir y adaptar fácilmente entre la interfaz Web del cuidador y la Aplicación Móvil, acelerando el tiempo de desarrollo y garantizando la consistencia visual.
- **Manipulación Eficiente del DOM Virtual y Estado en Tiempo Real:** Fundamental para reflejar de manera inmediata los cambios de telemetría en los mapas de calor y las alertas del sistema. React actualiza de forma óptima únicamente los elementos de la pantalla que cambian ante un evento (como la recepción de una nueva coordenada vía WebSockets), evitando recargas innecesarias y reduciendo el consumo de recursos en el dispositivo.
- **Ecosistema Nativo y Wearable Eficiente (React Native):** Al utilizar puentes nativos (*native bridges*), permite compilar código JavaScript directamente en componentes nativos de iOS y Android. Esto proporciona acceso directo a las API de hardware críticas del teléfono y del Smartwatch, tales como los sensores GPS en segundo plano, la vibración háptica para alertas de riesgo, y el control periférico de la cámara y el micrófono para la Grabación Silenciosa.

CAPÍTULO V: ESQUEMA Y ESTRATEGIA DE PRUEBAS

El esquema de pruebas para la validación del proceso web asegura la estabilidad antes de la liberación en producción, dividiéndose en múltiples niveles de pruebas tácticas operadas mediante herramientas de automatización.

Tipo de Prueba		Objetivo Tecnológico	Estrategia / Caso Práctico en Guardian of Missing
Pruebas Unitarias (Original)		Validar el comportamiento aislado de funciones, clases o algoritmos de código sin dependencias externas.	Verificar de forma lógica y matemática que el algoritmo de Geocercas dispare correctamente un booleano positivo (true) cuando una coordenada GPS simulada se encuentre fuera del radio límite del polígono establecido para el Módulo 1.
Pruebas Unitarias (Nueva: Reglas BFF)		Verificar el correcto filtrado, poda y formateo de estructuras de datos en capas intermedias aisladas.	Probar la función purificadora del componente BFF-Wearable. El test debe asegurar que al recibir un JSON de alerta de 50 campos desde el Backend Core, la función lo transforme de forma aislada en un payload ultra compacto de solo 4 campos esenciales (id, tipo_riesgo, color, coordenadas_cortas) para proteger la memoria RAM del Smartwatch.
Pruebas Unitarias (Nueva: Patrón)		Validar la integridad y tipado estricto en la instanciación automática de objetos de	Evaluar que la clase CreadorDeReportes (Factory Method) rechace y arroje una excepción si un payload para el ReporteModulo2 (Anónimo)

Factory)	negocio.	intenta adjuntar datos de identidad del usuario, garantizando las reglas de anonimato en memoria antes de tocar la base de datos.
Pruebas de Integración (Enfoque Bottom-Up)	Validar la comunicación del sistema probando primero los componentes de bajo nivel (BD, drivers) e ir ascendiendo hacia las API de alto nivel.	Estrategia Ascendente: Primero se testea que el driver geoespacial inserte coordenadas crudas en MongoDB de forma correcta. Tras validar esto, se integra con el servicio de cálculo de <i>Polling</i> de FastAPI. Finalmente, se conecta con la capa superior (Endpoints públicos), asegurando que los datos de geolocalización suban e impacten de forma limpia desde la base de datos hacia arriba.
Pruebas de Integración (Enfoque Top-Down)	Validar la arquitectura desde los niveles superiores (Interfaces/BFF) hacia abajo, simulando las capas inferiores aún no desarrolladas mediante <i>stubs</i> o <i>mocks</i> .	Estrategia Descendente: Probar la lógica de la plataforma Web del cuidador a través del BFF-Web. Se invoca el método de consolidación de alertas de la Fachada y, utilizando un <i>stub</i> (simulador estático) que suplanta a la base de datos MySQL de historial de auditoría, se comprueba que el BFF sea capaz de estructurar las líneas de tiempo de emergencia de forma correcta en la UI.
Pruebas de Integración (Original)	Comprobar la correcta sincronización, flujo de datos e intercomunicación entre los diferentes módulos, microservicios y APIs.	Verificar que al enviar una petición POST al endpoint del Botón de Pánico , el sistema backend orqueste de forma atómica la persistencia del registro en MySQL, almacene la sesión en caché con Redis y dispare eficazmente los tokens hacia Firebase Cloud Messaging (FCM).
Pruebas de Caja Negra (Original)	Validar flujos funcionales y de negocio completos desde la perspectiva del usuario final, sin inspeccionar la estructura del código interno.	Simular el recorrido del usuario en la App Móvil: presionar el Botón de Pánico en la pantalla, verificar visualmente que se active la interfaz de emergencia y confirmar que los contactos de confianza reciban la alerta en tiempo real en la Plataforma Web.
Pruebas de Sistema (E2E - Flujo de Emergencia)	Garantizar el correcto funcionamiento de la cadena completa de hardware, conectividad inalámbrica y software en un ciclo de vida real.	Ciclo Completo: Un usuario presiona el Botón de Pánico físico en el Smartwatch . Se prueba que viaje la orden vía Bluetooth al Teléfono Móvil , que el celular active la cámara trasera e inicie la Grabación Silenciosa enviando el flujo multimedia a AWS S3 en segundo plano, mientras el Backend distribuye alertas por

		WebSockets que hacen parpadear en rojo el panel de control en la Plataforma Web del cuidador en tiempo real.
Pruebas de Sistema (E2E - Alerta de Geocerca)	Evaluar la reacción autónoma del ecosistema completo ante eventos desencadenados por sensores en segundo plano.	Ciclo Completo: Utilizando un emulador en movimiento, se simula que el dispositivo móvil de un paciente con Alzheimer sale del polígono de rango seguro. El sistema en segundo plano detecta la telemetría, el backend procesa la ruptura mediante el patrón Observer y el celular del familiar/cuidador recibe una notificación push de alta prioridad con la Última Ubicación Conocida en menos de 2 segundos.
Pruebas de Seguridad (Original)	Garantizar la protección de datos personales altamente sensibles (ubicación, identidad, logs) y blindar la integridad del sistema contra accesos no autorizados.	Simular ataques de inyección al endpoint de Reportes Ciudadanos Anónimos o intentar realizar peticiones HTTP de geolocalización sin una cabecera válida de autenticación (Bearer Token), asegurando que el servidor responda estrictamente con un error 401 Unauthorized.

ESQUEMA DE PRUEBAS:

		Tipo de Prueba		Objetivo Tecnológico		Estrategia / Caso Práctico...		
1		Pruebas Unitarias (Original)		Validar el comportamiento...		Verificar de forma lógica y...		
2		Pruebas Unitarias (Nueva:...		Verificar el correcto filtrado,...		Probar la función purificadora...		
3		Pruebas Unitarias (Nueva:...		Validar la integridad y tipado...		Evaluar que la clase...		
4		Pruebas de Integración...		Validar la comunicación del...		Estrategia Ascendente: Prime...		
5		Pruebas de Integración...		Validar la arquitectura desde...		Estrategia Descendente: Prob...		
6		Pruebas de Integración...		Comprobar la correcta...		Verificar que al enviar una...		
7		Pruebas de Caja Negra...		Validar flujos funcionales y de...		Simular el recorrido del usar...		
8		Pruebas de Sistema (E2E - Fluj...		Garantizar el correcto...		Ciclo Completo: Un usuario...		
9		Pruebas de Sistema (E2E - ...		Evaluar la reacción autónoma...		Ciclo Completo: Utilizando un...		
10		Pruebas de Seguridad...		Garantizar la protección de...		Simular ataques de inyección...		

CAPÍTULO VI: REPORTE DE CONFIGURACIÓN DEL ENTORNO DE DESARROLLO

Este capítulo detalla los componentes de software, herramientas periféricas y la matriz de parametrización requerida para homologar las estaciones de trabajo del equipo de desarrollo, garantizando la replicabilidad del ecosistema (Smartwatch, App Móvil, Web y Backend) en entornos locales.

6.1 Lista de Herramientas del Entorno

Para asegurar que tanto el Frontend multiplataforma como el Backend asíncrono coexistan sin conflictos de dependencias, se establece la siguiente pila de herramientas obligatoria:

Categoría	Herramienta / Tecnología	Versión Mínima	Propósito en el Proyecto
Entorno Backend	Python	3.10.x	Lenguaje core para el desarrollo de las API asíncronas con FastAPI.
Gestor de Dependencias	Poetry / venv	Actual	Aislamiento riguroso de librerías de Python para evitar el conflicto de
Categoría	Herramienta / Tecnología	Versión Mínima	Propósito en el Proyecto
			entornos globales.
Entorno Frontend	Node.js (Runtime)	18.x LTS	Motor de ejecución para compilar y ejecutar el ecosistema React y React Native.
Persistencia Relacional	MySQL Server	8.0	Almacenamiento de datos de alta integridad (usuarios, roles, contactos de confianza).
Persistencia No Relacional	MongoDB Community Server	6.0	Base de datos geoespacial para el histórico de telemetría y reportes anónimos.

Caché en Memoria	Redis Server	7.0	Gestión de sesiones de pánico activas y canales de comunicación WebSockets rápidos.
Emulación de Dispositivos	Android Studio / Xcode	Actual	Entornos IDE nativos para desplegar emuladores móviles y simuladores de Smartwatch (Wear OS / watchOS).
Control de Versiones	Git	2.x	Control de versiones distribuido bajo la estrategia de ramificación del equipo.
Cliente de Pruebas	Postman / Insomnia	Actual	Suite de pruebas para disparar payloads JSON ficticios y simular peticiones de emergencia.

6.2 ¶¶ Parámetros de Configuración del Entorno (Archivo .env)

Para garantizar la portabilidad, el desacoplamiento de la infraestructura y el cumplimiento de las normativas de seguridad básicas, todas las credenciales se inyectan en el runtime del sistema mediante un archivo de variables de entorno local.

A continuación, se define la estructura paramétrica del archivo `.env` modelo que debe configurarse en la raíz del proyecto Backend:

Fragmento de código

```
#
=====
# --- CONFIGURACIÓN DEL SERVIDOR WEB CORE ---
# =====
APP_ENV=development
SECRET_KEY=9a82f3b4c5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2
ACCESS_TOKEN_EXPIRE_MINUTES=60

#
=====
# --- PERSISTENCIA RELACIONAL (MYSQL - CONTROL DE IDENTIDADES) ---
# =====
MYSQL_HOST=127.0.0.1
MYSQL_PORT=3306
MYSQL_USER=mr_dev_user
MYSQL_PASSWORD=DB_Secure_Pass_2026
MYSQL_DATABASE=guardian_missing_relational

#
=====
# --- PERSISTENCIA NO RELACIONAL / TELEMETRÍA (MONGODB - INDICES ESPACIALES) ---
# =====
MONGO_URI=mongodb://mr_dev_user:Mongo_Secure_Pass_2026@127.0.0.1:27017/
MONGO_DATABASE=guardian_missing_telemetry

#
=====
# --- CACHÉ EN MEMORIA Y ESTADOS DE EMERGENCIA (REDIS CORE) ---
# =====
REDIS_HOST=127.0.0.1
REDIS_PORT=6379

#
=====
# --- PARAMETRIZACIÓN DE REGLAS DE NEGOCIO ---
# =====
# Frecuencia en segundos de la captura de GPS para usuarios vulnerables (Módulo 1)
DEFAULT_POLLING_INTERVAL_SECONDS=60

# Radio máximo permitido para delimitar zonas seguras operativas
MAX_GEOFENCE_RADIUS_METERS=5000

# Umbral de coincidencia/score mínimo del algoritmo para validar reportes ciudadanos
```


MODERATION_ALGORITHM_THRESHOLD=0.75

#

=====

--- SERVICIOS DE INFRAESTRUCTURA Y COMPONENTES EXTERNOS ---

=====

API key para la distribución de notificaciones push de pánico hacia móviles y relojes

FIREBASE_API_KEY=AlzaSyA1B2C3D4E5F6G7H8I9J0K1L2M3N4O5P6

Ruta del certificado SSL local obligatorio para habilitar conexiones seguras WSS (WebSockets)

SSL_CERT_PATH=/etc/ssl/certs/guardian_missing_dev.crt

